

Esame Programmazione di Sistema / API Programming

Data: 2021-10-18

THEORETICAL AND PROGRAMMING QUESTIONS

18 ottobre 2021

La struttura generica `Processor<T>` consente a un insieme di thread produttori di inviare oggetti istanza del tipo `T` (che si assume copiabile) a un thread consumatore, il cui comportamento è definito tramite una funzione fornita come parametro del costruttore.

Il costruttore di tale struttura riceve, come parametro, una funzione (o una closure) che accetta un argomento di tipo `T` e restituisce `()`. La struttura fornisce una coda per gestire gli oggetti inviati dai produttori e offre i seguenti metodi:

1. **Metodo** `send(&self, item: T)`: Permette ai produttori di sottomettere un oggetto da elaborare. L'oggetto viene inserito in una coda in attesa che il thread consumatore esterno lo elabori. Se il metodo `close(...)` è stato invocato, eventuali chiamate a `send(...)` devono generare un errore o produrre un comportamento indefinito, a scelta dell'implementazione.
2. **Metodo** `close(&self)`: Segnala la fine dell'accettazione di nuovi elementi. Dopo l'invocazione di questo metodo:
 - Non sarà più possibile inviare nuovi dati tramite `send(...)`.
 - Il metodo non ritorna fino a quando la coda non è vuota e tutte le operazioni di elaborazione sono state completate.

La struttura `Processor<T>` deve garantire la sincronizzazione tra i produttori e il consumatore esterno utilizzando primitive di sincronizzazione di Rust. La logica di elaborazione e il ciclo di vita del thread consumatore devono essere gestiti esternamente.

Viene fornita la seguente dichiarazione di struct Rust come punto di partenza:

```
1 impl<T: Send + 'static> Processor<T> {
2     fn new<F>(f: F) -> Self
3     where F: Fn(T) + Send + 'static {}
4
5     fn send(&self, item: T) {}
6
7     fn close(&self) {}
8 }
```

ANSWERS

Original Solution Document

Overview

This document serves as a placeholder for the step-by-step solution to the exam dated 2021-10-18.

Step-by-Step Solution

Step 1: Problem Analysis

[Analyze the concurrency problem described in the exam and identify the necessary data structures and synchronization primitives.]

Step 2: Architecture Design

[Outline the Rust structures, traits, and threading model needed to satisfy the requirements.]

Step 3: Implementation

```
1 // Placeholder for Rust implementation
2 fn main() {
3     println!("Solution pending...");
4 }
```

Step 4: Testing and Verification

[Describe how to test the implementation to ensure it meets the concurrency and performance constraints.]

Proposed Working Solution

Nonostante il documento originale contenga solo un placeholder, forniamo di seguito un'implementazione completa in Rust basata sul testo dell'esame.

Analisi e Design

La struttura `Processor<T>` funge da ponte tra thread produttori e un singolo thread consumatore. Per inserire oggetti nella coda in modo sicuro e per notificare il thread consumatore di nuovi arrivi (o della chiusura), una soluzione efficiente è utilizzare un canale MPSC (*Multi-Producer, Single-Consumer*) della libreria standard `std::sync::mpsc`.

Per permettere la condivisione della struttura `Processor<T>` tra vari produttori tramite riferimenti condivisi (e.g. all'interno di `Arc`), la chiusura di `send` richiede una mutabilità interiore (*interior mutability*) per il `Sender` e per il `JoinHandle`. Usiamo quindi `Mutex<Option<Sender<T>>>` e `Mutex<Option<JoinHandle<()>>>`.

- **new**: Crea il canale, fa spawn del thread consumatore che itera sugli elementi ricevuti e applica la closure `f`, e salva sia il `Sender` che il `JoinHandle`.
- **send**: Blocca il `Mutex` e, se il `Sender` è ancora presente (non chiuso), invia il dato. In caso contrario (se `close` è stato invocato), genera un *panic* come comportamento indefinito ammesso.
- **close**: Rimuove il `Sender` dal `Mutex` e lo droppa (sbloccando il thread consumatore alla fine degli elementi), dopodiché estrae il `JoinHandle` e ne attende la terminazione tramite `join()`. Questo garantisce l'attesa del termine dell'elaborazione di tutti i messaggi presenti nella coda.

Implementazione Rust

```
1 use std::sync::{mpsc, Mutex};
2 use std::thread::{self, JoinHandle};
3
4 pub struct Processor<T> {
5     sender: Mutex<Option<mpsc::Sender<T>>>,
6     handle: Mutex<Option<JoinHandle<()>>>,
7 }
8
9 impl<T: Send + 'static> Processor<T> {
10     pub fn new<F>(f: F) -> Self
11     where
12         F: Fn(T) + Send + 'static,
13     {
14         let (tx, rx) = mpsc::channel();
15
16         let handle = thread::spawn(move || {
17             for item in rx {
18                 f(item);
19             }
20         });
21
22         Self {
23             sender: Mutex::new(Some(tx)),
24             handle: Mutex::new(Some(handle)),
25         }
26     }
27
28     pub fn send(&self, item: T) {
29         let mut guard = self.sender.lock().unwrap();
30         if let Some(ref tx) = *guard {
31             // Invio l'elemento e ignoro eventuali errori se il
ricevitore e' morto inaspettatamente
32             let _ = tx.send(item);
33         } else {
34             panic!("Cannot send: the processor has been closed.");
35         }
36     }
37
38     pub fn close(&self) {
39         // Drop del sender: segnala la fine della coda al consumatore
40         {
41             let mut sender_guard = self.sender.lock().unwrap();
42             sender_guard.take();
43         }
44
45         // Join del thread consumatore per attendere la fine dell'
elaborazione
46         let mut handle_guard = self.handle.lock().unwrap();
47         if let Some(handle) = handle_guard.take() {
48             let _ = handle.join();
49         }
50     }
51 }
```